

Trabalho Prático de Agentes: Contract Net Protocol

Felipe Escobar Koizimi

Sistemas Multiagentes – Prof. Jomi F. Hübner

Há n agentes que precisam contratar um serviço (os *initiators*) e m agentes que oferecem esse serviço (os *participants*). Cada initiator faz i contratações ao mesmo tempo, com n entre 2 e 199, m entre 2 e 49 e i entre 1 e 9. Todos os participants oferecem um único serviço, mas cada um define o seu preço conforme a sua estratégia.

Para resolver o problema, o Contract Net Protocol (CNP) foi implementado em Jason 3.3, com os agentes escritos em AgentSpeak e o ambiente em Java. O ambiente não participa das negociações: ele apenas registra os eventos e encerra a execução quando todas terminam. Também foram desenvolvidos scripts para executar os testes e os experimentos.

Seção	Conteúdo
1	O protocolo
2	Modelagem
3	Método dos experimentos
4–6	Experimentos 1 a 3: variação de n , m e i
7	Experimento 4: estresse
8	Avaliação da abordagem de agentes
9	Avaliação das ferramentas
10	Critério de comparação entre implementações
11	Conclusões

O código, os scripts e os dados usados neste relatório estão em <https://github.com/escobar-felipe/tp-cnp-jason>.

1. O protocolo

Uma negociação do CNP tem cinco passos:

```
initiator                                participants
|--- cfp(Cid, servico_a) ----->|   (para todos)
|<-- propose(Cid, Preco) -----|   ou refuse(Cid)
|   espera todos responderem e escolhe o menor preco
|--- accept_proposal(Cid) ----->|   (vencedor)
|--- reject_proposal(Cid) ----->|   (os outros)
|<-- inform_done(Cid) -----|   (vencedor, depois de executar)
```

O initiator anuncia o serviço para todos os participants (**cfp**, de *call for proposals*). Cada um responde com um preço ou com uma recusa. Quando todos respondem, ou quando o prazo de 2000 ms termina, o initiator escolhe o menor preço, informa o vencedor e rejeita as demais propostas. O vencedor executa o serviço e informa a conclusão.

Quando os m participants respondem, uma negociação troca m mensagens de **cfp**, m de **propose**, 1 de **accept_proposal**, $m - 1$ de **reject_proposal** e 1 de **inform_done**, ou seja, $3m + 1$ mensagens. Uma rodada completa troca $n \times i \times (3m + 1)$. Essa fórmula foi usada para verificar a contagem feita durante as execuções.

2. Modelagem

2.1 Agentes e o sistema

O sistema é descrito no `cnp.mas2j`:

```
MAS cnp {
  environment: cnp.CnpEnv(2, 3, 2)
  agents:
    initiator    [beliefs="tarefas(3)"] #2;
    participant   #2;
    aslSourcePath: "src/agt";
}
```

O #2 cria duas instâncias de cada agente, e o Jason numera os nomes: `initiator1`, `initiator2`, `participant1` e `participant2`. Cada initiator começa com a crença `tarefas(3)`, que é o i . O ambiente recebe (n, i, m) para saber quantas negociações esperar.

Os agentes seguem a divisão do AgentSpeak em crenças, regras e planos. O participant sabe que serviço oferece (`servico(servico_a)`) e deduz, por regras, o próprio número e a sua estratégia. O initiator começa com o objetivo `!start` e trata as mensagens recebidas com planos.

2.2 Negociações em paralelo

O initiator abre as i negociações assim:

```
+!start : tarefas(I)
  <- .wait(1000);
  for (.range(K, 1, I)) {
    !!cnp(K)
  }.
```

O `.wait(1000)` garante que todos os agentes já tenham sido criados. O ponto principal é o operador `!!`. Com `!cnp(K)`, cada negociação seria um subobjetivo da intenção atual, e o initiator só abriria a segunda depois de terminar a primeira. Com `!!cnp(K)`, cada negociação torna-se uma intenção independente, e as i negociações executam ao mesmo tempo.

Como as negociações executam ao mesmo tempo, elas precisam de um identificador. Cada uma recebe um `Cid = c(Me, K)`, por exemplo `c(initiator1, 3)` para a terceira negociação do initiator1. Toda mensagem e toda crença de uma negociação carregam o seu `Cid`. Sem isso, a resposta de uma negociação poderia ser tomada como resposta de outra.

2.3 Estados de uma negociação

Cada negociação passa pelos estados abaixo, guardados na crença `estado(Cid, X)`:

```
coletando --> decidir --> sem_proposta      (ninguem propos)
                    +-> esperando +-> concluida      (inform_done do vencedor)
                    +-> timeout_resultado (5 s sem entrega)
```

O initiator decide assim que todos os participants respondem. Cada proposta ou recusa recebida dispara um plano que verifica se todas já chegaram:

```
todos_responderam(Cid) :-
    cfps(Cid, NP) &
    .count(propose(Cid, _)[source(_)], NA) &
    .count(refuse(Cid)[source(_)], NR) &
    NA + NR >= NP.

+propose(Cid, _) : estado(Cid, coletando) & todos_responderam(Cid)
<- !decidir(Cid).
```

Há um plano equivalente para `refuse`. O prazo de 2000 ms continua ativo, mas só é usado quando algum participant não responde. Sem ele, o initiator aguardaria indefinidamente.

Os planos que decidem e que encerram uma negociação são marcados como `[atomic]`. Enquanto um deles executa, nenhuma outra intenção do initiator é executada. Assim, uma negociação nunca termina duas vezes: ou chega o `inform_done` a tempo, ou o prazo expira.

Para trocar o estado, foi usado:

```
+!mudar_estado(Cid, Novo)
  <- -estado(Cid, _);
  +estado(Cid, Novo).
```

A forma mais curta seria `++estado(Cid, Novo)`, mas o `++` do Jason remove a primeira crença `estado(_, _)` que encontrar, e ela pode ser de outra negociação do mesmo initiator. Com várias negociações simultâneas, isso corromperia os estados.

O initiator também verifica a origem de cada mensagem. A entrega só é aceita se vier do vencedor e se a negociação ainda estiver esperando:

```
+inform_done(Cid)[source(W)] : vencedor(Cid, W, Preco) & estado(Cid, esperando)
```

O participant, por sua vez, só executa o serviço se tiver feito proposta para aquele initiator naquela negociação, e uma única vez.

2.4 Estratégias de preço

A estratégia de cada participant é definida pelo seu número, pelo resto da divisão por 3:

Número mod 3	Estratégia	Preço
0	fixa	sempre 1000
1	agressiva	sorteado entre 900 e 1050
2	aleatória	sorteado entre 800 e 1200

Os intervalos se sobrepõem, portanto qualquer estratégia pode vencer. A fixa é previsível. A

agressiva quase sempre fica abaixo da fixa, mas nunca desce de 900. A aleatória pode ser a mais barata ou a mais cara de todas. O preço é sorteado novamente a cada pedido. O initiator escolhe o menor preço, e o empate é decidido pelo menor nome. Isso decorre do formato das propostas: o initiator reúne todas em uma lista de `of(Preço, Nome)`, e o `.min` do Jason compara primeiro o preço e depois o nome.

2.5 Ambiente e contagem de mensagens

O ambiente (`CnpEnv.java`) só aceita uma ação, `registrar`. Os initiators a chamam quando abrem uma negociação e quando ela termina. Cada chamada gera uma linha em `results/eventos.csv`, com o horário em milissegundos, o agente, o evento, o `Cid` e, nas conclusões, o vencedor e o preço. O método é `synchronized`, porque muitos agentes registram ao mesmo tempo. Quando o número de negociações terminadas chega a $n \times i$, o ambiente grava um `resumo.csv` e encerra o processo.

As mensagens são contadas pelo próprio initiator, em cada negociação. No momento da decisão, ele soma os `cfps` que enviou com as propostas e as recusas que chegaram no prazo:

```
.count(refuse(Cid)[source(_)], NR);
.length(L, NL);
?cfps(Cid, NP);
T = NP + NL + NR;
+msgs(Cid, T);
```

Depois acrescenta o aceite e as rejeições, uma mensagem para cada proposta recebida, e, se o vencedor entregar, mais uma do `inform_done`. O total vai no último argumento do `registrar` do estado final, e o ambiente soma os totais de todas as negociações.

Essa contagem considera apenas o que acontece dentro da negociação. Uma proposta que chega depois da decisão, ou um `inform_done` que chega depois do timeout, não é contabilizada.

3. Método dos experimentos

Foi variado um parâmetro de cada vez, com 3 repetições por cenário:

Bloco	n	m	i
Variar n	2, 10, 50, 100, 199	10	3
Variar m	50	2, 5, 10, 25, 49	3
Variar i	50	10	1, 3, 5, 9
Estresse	199	49	9

O cenário 50/10/3 aparece nos três primeiros blocos e foi executado uma única vez. São 13 cenários e 39 execuções.

As métricas, calculadas a partir do `eventos.csv` de cada execução, são:

Métrica	Como é calculada
Tempo total	da primeira abertura até o último estado final
Latência	da abertura até o estado final de cada negociação (média)
Vazão	negociações concluídas por segundo de tempo total
Taxa de sucesso	concluídas $\div (n \times i)$
Mensagens	soma das mensagens de cada negociação, contadas pelo initiator
Memória	pico de memória do processo, medido pelo <code>/usr/bin/time</code> -1

A latência inclui os 100 ms que o vencedor leva para executar o serviço. O restante é o tempo gasto com as mensagens e com o raciocínio dos agentes.

O tempo total não inclui a inicialização da JVM nem o segundo de espera do `!start`, porque começa a contar na primeira abertura.

4. Experimento 1: variação de n

Com $m = 10$ e $i = 3$:

n	Tempo total (ms)	Desvio (ms)	Latência (ms)	Vazão (/s)	Mensagens	Memória (MB)
2	145	3	137	41,4	186	94
10	205	1	177	146,6	930	122
50	666	113	572	229,3	4.650	184
100	849	241	749	372,4	9.300	336
199	1.919	179	1.651	312,8	18.507	507

Comentário

Todas as negociações foram concluídas, e o número de mensagens coincidiu com a fórmula em todos os cenários. A latência subiu de 137 ms com 2 initiators para 1,7 s com 199.

A vazão cresceu até 100 initiators e caiu com 199, de 372,4 para 312,8 negociações por segundo. A partir desse ponto, mais initiators apenas aumentam o tempo de espera. A memória cresceu cerca de 2,1 MB por agente.

5. Experimento 2: variação de m

Com $n = 50$ e $i = 3$:

m	Tempo total (ms)	Desvio (ms)	Latência (ms)	Vazão (/s)	Mensagens	Memória (MB)
2	267	3	223	562,5	1.050	148
5	338	10	277	443,6	2.400	175
10	666	113	572	229,3	4.650	184
25	1.010	48	887	148,7	11.400	312
49	1.815	312	1.690	84,4	22.200	447

Comentário

Cada negociação troca $3m + 1$ mensagens, portanto o m é o parâmetro de maior impacto por negociação. De 2 para 49 participants, as mensagens aumentaram 21 vezes, a latência 7,6 vezes, e a vazão caiu de 562,5 para 84,4 negociações por segundo.

6. Experimento 3: variação de i

Com $n = 50$ e $m = 10$:

i	Tempo total (ms)	Desvio (ms)	Latência (ms)	Vazão (/s)	Mensagens	Memória (MB)
1	249	9	216	201,0	1.550	157
3	666	113	572	229,3	4.650	184
5	847	5	733	295,2	7.750	292
9	1.349	220	1.143	339,7	13.950	425

Comentário

As negociações de cada initiator executaram em paralelo, mas concorrem pelos mesmos participants. Com 9 vezes mais negociações, a vazão aumentou apenas 1,7 vezes, e a latência passou de 216 ms para 1,1 s.

Considerando os Experimentos 1 e 3, a latência acompanha o total de negociações ($n \times i$), e não a forma como elas se dividem entre os initiators:

Negociações	Cenário	Latência (ms)
150	50×3	572
250	50×5	733
300	100×3	749
450	50×9	1.143
597	199×3	1.651

A Figura 1 mostra a latência nos três experimentos.

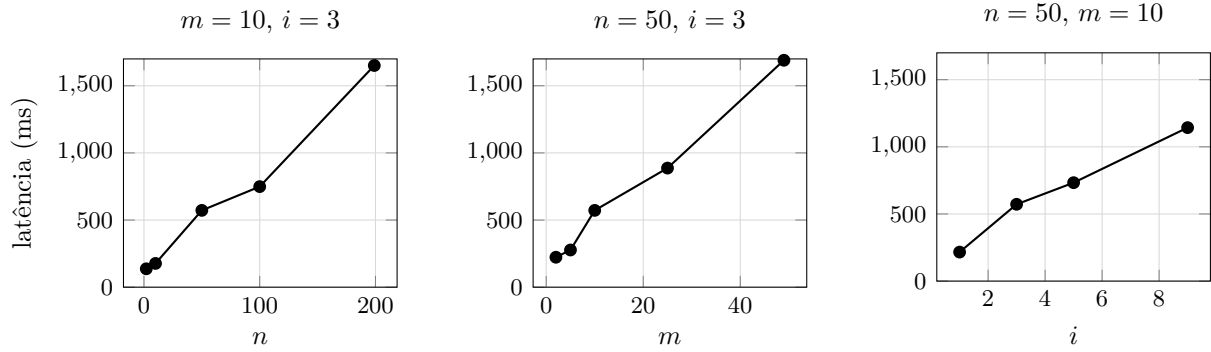


Figura 1: Latência média por negociação em cada experimento. Média de 3 repetições.

7. Experimento 4: estresse

Com $n = 199$, $m = 49$ e $i = 9$, são 248 agentes e 1.791 negociações por execução:

Repetição	Concluídas	Timeout	Taxa de sucesso	Mensagens	Tempo total (ms)	Memória (MB)
1	1.644	147	91,8%	250.719	24.506	1.043
2	1.539	252	85,9%	202.860	26.423	1.255
3	1.464	327	81,7%	262.621	34.874	1.102

A fórmula prevê 265.068 mensagens por execução.

Comentário

Todas as 1.791 negociações terminaram, e de 81,7% a 91,8% foram concluídas. As falhas foram todas `timeout_resultado`. O sistema saturou: as negociações levaram de 10 a 35 s, e a lentidão fez com que vários prazos de 2 e de 5 segundos expirassem.

O número de mensagens ficou abaixo da fórmula porque, depois de um prazo expirado, as respostas atrasadas não são contabilizadas. Os contratos ficaram distribuídos entre 25 e 35 participants por repetição.

8. Avaliação da abordagem de agentes

Pontos positivos:

- Cada papel do CNP corresponde a um tipo de agente, e cada passo do protocolo a uma mensagem. Não foi necessário nenhum coordenador central.
- Cada agente armazena apenas o que precisa nas suas crenças. As verificações do protocolo, como aceitar a entrega só se vier do vencedor, ficaram com uma linha cada.

Dificuldades encontradas:

- A concorrência continuou exigindo cuidado. Foi necessário marcar alguns planos como `[atomic]` e trocar o `-+estado`, como explicado na Seção 2.3.

- O Jason agrupa propostas iguais em uma única crença, com várias fontes. Para verificar se todos já haviam respondido, foi necessário contar as fontes (`[source(_)]`), e não as crenças.

9. Avaliação das ferramentas

O Jason mostrou-se adequado para o problema. Os dois agentes têm, juntos, 102 linhas de código, e as ações internas `.send`, `.findall`, `.count`, `.min` e `.wait` atenderam às necessidades do protocolo. O ambiente em Java exigiu apenas dois métodos, `init` e `executeAction`.

10. Critério de comparação entre implementações

Para comparar esta implementação com outra, são propostas duas etapas.

A primeira é a correção: a implementação deve passar nos seis cenários de teste do repositório (`scripts/testes.py`).

A segunda é a eficiência, medida na mesma máquina e nos mesmos cenários:

Medida	Cenário	Esta implementação
Latência por negociação	50, 10, 3	572 ms
Latência por negociação	199, 10, 3	1.651 ms
Mensagens por negociação	50, 10, 3	31 ($= 3m + 1$)
Memória por agente	variação de n	2,1 MB
Taxa de sucesso	199, 49, 9	86,5% (de 81,7% a 91,8%)

A latência pode ser comparada diretamente porque o initiator decide assim que todos respondem, sem esperar um prazo fixo. Em uma implementação que sempre espera o prazo, esse prazo deve ser descontado antes da comparação.

11. Conclusões

O trabalho implementou o Contract Net Protocol em Jason e funcionou como esperado. As negociações rodaram em paralelo, cada participant usou a sua estratégia de preço, e o sistema chegou a 597 negociações ao mesmo tempo, com 209 agentes, sem perder nenhuma. Fora do cenário de estresse, todas as negociações foram concluídas e o número de mensagens foi exatamente o que o protocolo prevê.

Nos experimentos, o tempo de cada negociação cresceu com os três parâmetros. O que mais pesou foi o total de negociações ($n \times i$). O m foi o que mais encareceu cada negociação, porque cada participant a mais significa mais mensagens.

No estresse, com 1.791 negociações, o sistema ficou lento, mas todas terminaram, e entre 81,7% e 91,8% foram concluídas.

O trabalho tem algumas limitações. Os prazos de 2 e de 5 segundos são fixos e não mudam com a carga, e foi isso que causou os timeouts no estresse. Os participants aceitam todos os contratos que recebem, sem limite. E as medições foram feitas em uma única máquina, com 3 repetições por cenário, o que é pouco para tirar conclusões mais fortes.